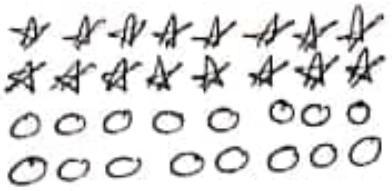
RBF (Radial Basis function)

Why RBF

→ Some datasets can not be seperated using a straight Line

Eg



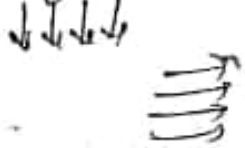
ie data is called non-linearly separable data

→ If data is not linearly seperable in current space, convert it into a higher-dimensional space, where it becomes seperable.

→ How RBF works

① Increase the dimensionality (2D-3D)
change dimensionality using requirement

② Expand the direction (Horizontal)
or
compress the direction (vertical)



③ changing direction is important & cannot do it randomly, we need to follow some functions as below (How RBF does this)

* consider one center randomly
* Draw concentric circles



④ measure distance from center & calculate for every data point distance from center

To expand / compress use ③ Radial function Basis

(a) Multiquadratic $\phi(r) = \sqrt{r^2 + c^2}$ $c > 0$ constant

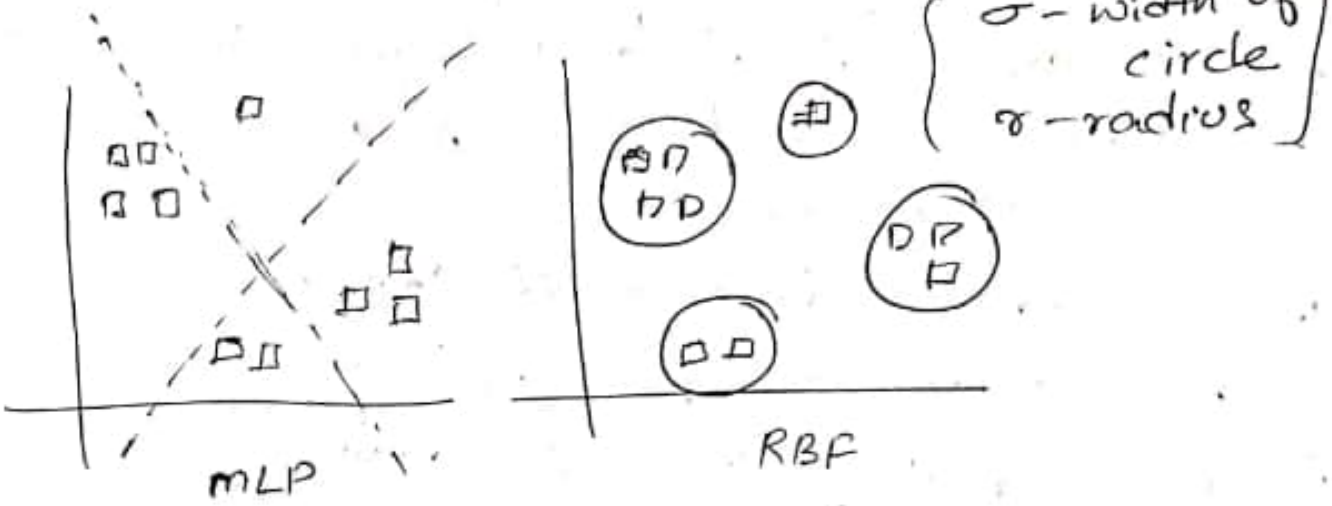
(b) Inverse Multiquadratic

$$\phi(r) = \frac{1}{\sqrt{r^2 + c^2}}$$

(3) Gaussian Function.

$$\phi(r) = \exp\left(\frac{-r^2}{2\sigma^2}\right)$$

σ - width of circle
r - radius



After applying RBF Data overlapping before now seperate closter

RBF Structure

I/p Layer - Takes i/p features $(x_1, x_2 \cdots x_n)$

Hidden Layer - RBF neurons

only one hidden Layer

Each neuron
- center
- apply Gaussian

o/p - combines hidden Layer output performs Linear Classification

Real Time Eg

Points near Radar - Enemy

Points far - not from radar enemy

distance decides classification

RBF Vs. MLP.

	MLP	RBF
Hidden Layers	— many	only one
decision boundary	— Linear	non-Linear
Activation	— Sigmoid	Gaussian
Training	— Backpropagation	Faster
Handles non Linear data	— Harder	Easier

Interpolation and Basis function

→ Interpolation means estimating unknown values using known data points

Eg: If you know at $x=1, y=2$, at $x=2, y=4$ then it will help you know y at $x=1.5$

→ Interpolations and Basis functions are important in function approximations and regression tasks

→ It is often used in ML to predict o/p's for new inputs based on the learned relationships

→ Types of Interpolations are

Linear Interpolation , Polynomial & Spline

- connects data points using straight lines.
- very simple and fast

But cannot model complex patterns

Polynomial Interpolation

- It uses polynomials to fit data
- can capture curved & complex patterns

But Higher degree polynomials cause Oscillation (overfitting)

Spline Interpolation (used in Computer Graphics, CAD, maths, engg)

- It uses piecewise polynomials.
- It ensures smoothness at datapoints avoiding oscillation problem for complex polynomials.
- Divides the data into smaller segments & fits a seperate polynomial to fit all datapoints.

Basis Functions

→ Basis Functions are a set of functions used to represent data in a new feature space

→ They transform the Original i/p features into a high dimensional space, making it easier to capture complex relationships.

→ It is mainly used in regression problems, where we want to model the relationship between i/p & o/p variables.

Types of Basis Functions commonly used –

① Polynomial Basis Function

- Represents data as polynomials of i/p variables

$$\phi(x) = [1, x, x^2, \ldots, x^d]$$

- captures non-linear patterns

② Radial Basis Function

- Functions that depend on distance from centre,
- commonly used in Gaussian processes and SVM.
- Eg- Image processing, computer Graphics Numerical analysis, Signal processing Geographical Info-systems. etc

$$\phi(x) = \frac{e^{-(x-c)^2}}{2\sigma^2}$$

③ Fourier Basis Functions

- It uses sine and cosine functions
- It is best for periodic data
- Mainly used in signal processing & time series applic

Curse of Dimensionality (attributes/features)

- "When the number of Features (P) becomes very large compared to the number of Samples (N) learning becomes difficult."

$$P \gg N$$

i.e Problem when working with high dim. data.

- If the features (attributes) exceed more than 100 or so, we call it as higher dimensional data.

(no. of attributes / features in dataset are called dimensions)

Few Problems are —

1. Analysing the dataset in pre-processing (data sparse)
2. Analysing / visualizing data to identify different patterns
3. the model becomes slow ie, takes lot of time and difficult to train the machine learning algorithm.
4. Model overfits (poor generalization)

"The curse of dimensionality refers to problems that arise when working with high-dimensional data, making learning inefficient and inaccurate"

Eg.

	P_1	P_2	P_3	P_4	P_5	P_6	features
observations n_1							$P > N$
n_2							
n_3							

Hard to describe the relationship b/w predictor variables & response variables

Solutions to C. of D

(1) convert / Reduce the high dimension data without loosing the essence / data; so that we can easily analyse data to find different patterns.

1) Dimensionality Reduction

- It is a process of reducing the no. of dimensions / features in a dataset while retaining main information.

→ Reducing the dimensions will help
  (1) Reduce the complexity of a model
  (2) Improve the performance of a learning problem
  (3) make visualization easy.

→ Techniques for Dimensionality Reduction
  (1) PCA (Principal Component Analysis)
  (2) Single value decomposition (SVD)
  (3) Linear discrimant Analysis (LDA)

* The complexity of the model increases with the no. of features, and becomes difficult to find a good solution.

→ high dimensional data can lead to overfitting

→ There are 2 approaches to dimensionality reduction
  - Feature Selection
  - Feature Extraction

Feature Selection

- It involves selecting a subset of the original features that are important to the problem
- It involves picking important features.
- If there are n, no. of features and we are intrested in some k, then we discard the other (n-k) dimensions.

Feature Extraction

→ It involves creating new features by combining or transforming the original features.

→ The goal is to create a set of features that captures the essence of the original data in a lower dimensional space.

Importance

we check whether a particular feature is producing any error or not. If it produces less error that feature is considered important to add in creating high dimensional to low dimensional data

how to measure the Error —

2 methods are there (in regression problems)

MSE (Mean Squared Error) RMSE (Root mean Squared Error)

Sum of overall datapoints squared by the difference b/w the predicted and actual target variables.

If $y_1, y_2 \cdots y_n$ are observed values &
$\hat{y}_1, \hat{y}_2 \cdots \hat{y}_n$ are predicted values, then

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

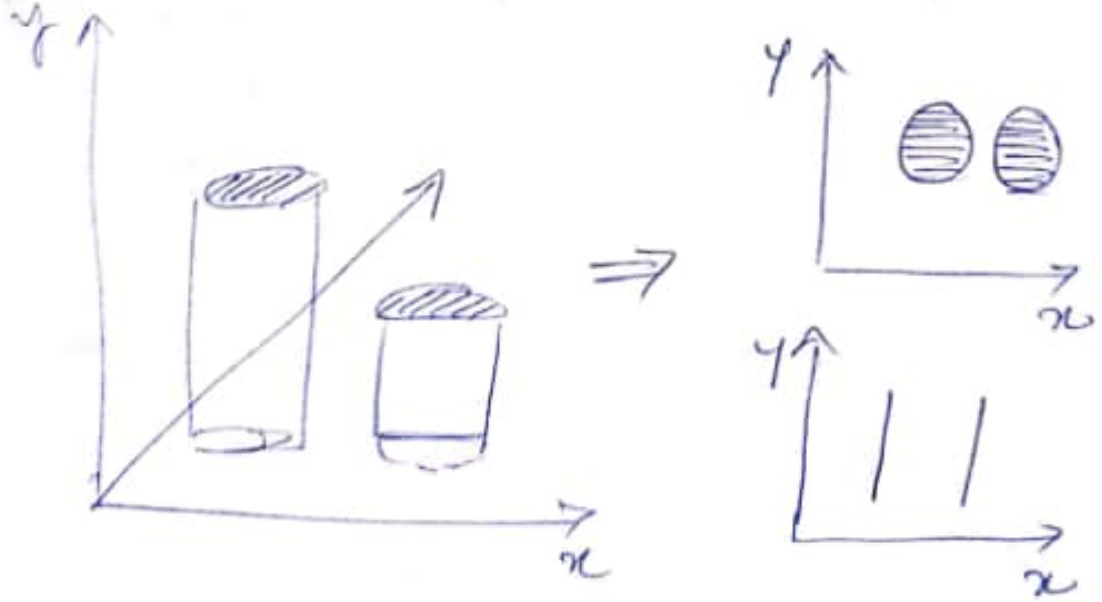
η = average of all the examples

n - no. of data points

y_i → actual target $\hat{y}_i$ – Predicted target

In classification problems,
we may cause the misclassification rate as a measure of the error.

$$\text{misclassification rate} = \frac{\text{no. of misclassified Examples}}{\text{total no. of examples}}$$



Dimensionality Reduction

Support Vector Machine

SVM is one of the most popular Supervised learning algorithms, which is used for classification as well as Regression problems.

- It learns from labelled data.

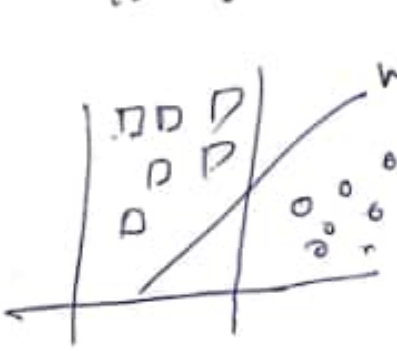
SVM tries to seperate data into classes using the best possible boundary.

Classification Problems are those where the target contains discrete number of possibilities

Email - spam / not spam

- The goal of the SVM algorithm is to create the best line as decision boundary that seperates classes with maximum distance between them..
- This best boundary is called a hyperplane..
- A hyperplane is the line / surface that seperates data points.





→ Support vectors - these are the points that are closer to the hyperplane

Margin - distance between the hyperplane & the nearest data points of either class.

SVM tries to maximize this margin to improve classification confidence.

Hard Margin – perfectly separable by hyperplane i.e. all points lie outside the margin.

Soft margin – misclassification / margin violation

Kernel function – transforms data into a higher dim to linearly separable.

If there are 2 features – hyperplane is a straight line
if 3 – hyperplane will be 2-dimensional plane

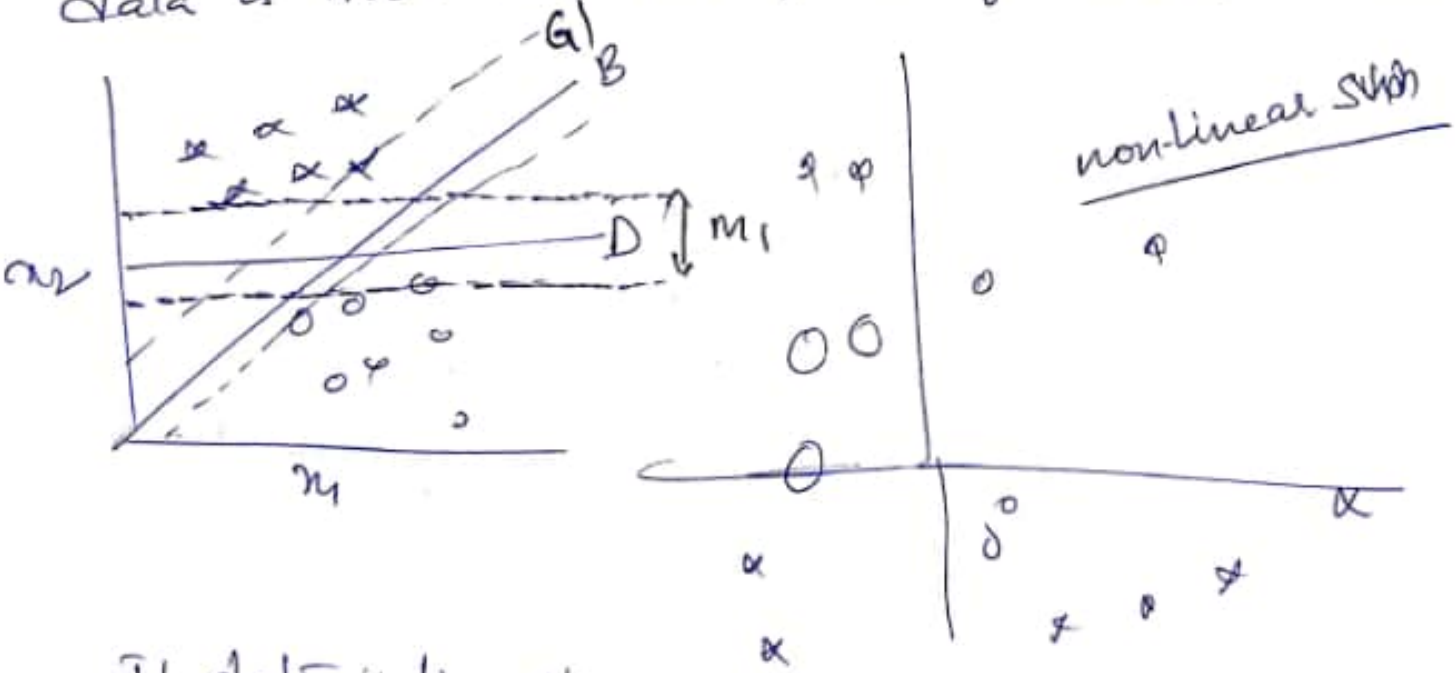
SVM – Two types < Linear SVM
Non-Linear SVM

Linear SVM

It is used for linearly separable data, ie classify the data using a straight-line. such data is termed as linearly separable data & the classifier is called Linear SVM Classifier.

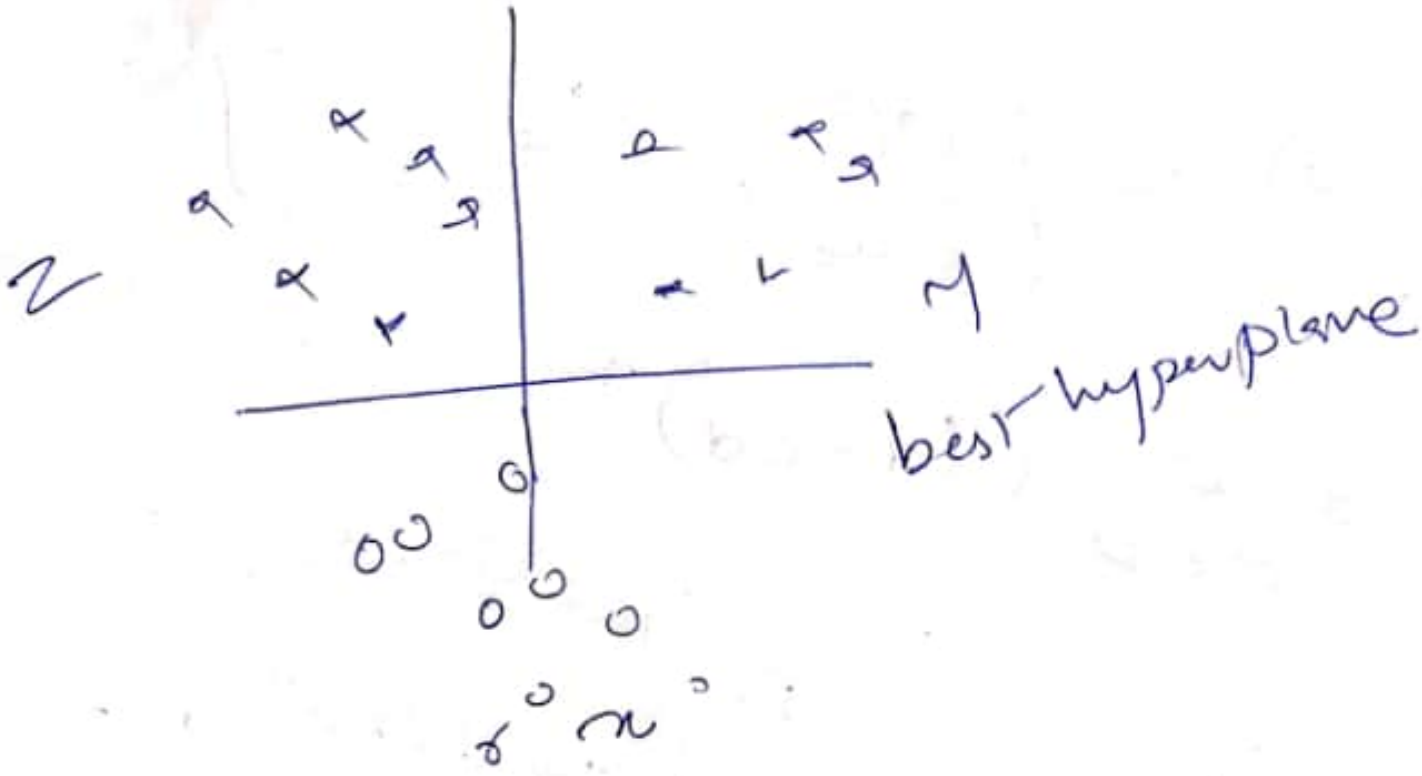
Nonlinear SVM

→ It is used for non-linearly separable data, & cannot be classified using a straight line. Hence that data is non-linear data & Classifier is non-linear SVM



If data is linearly arranged, we can separate it by using st. line, so to convert it we add one more dimensions, z.

then $z = x^2 + y^2$ → mapping function.

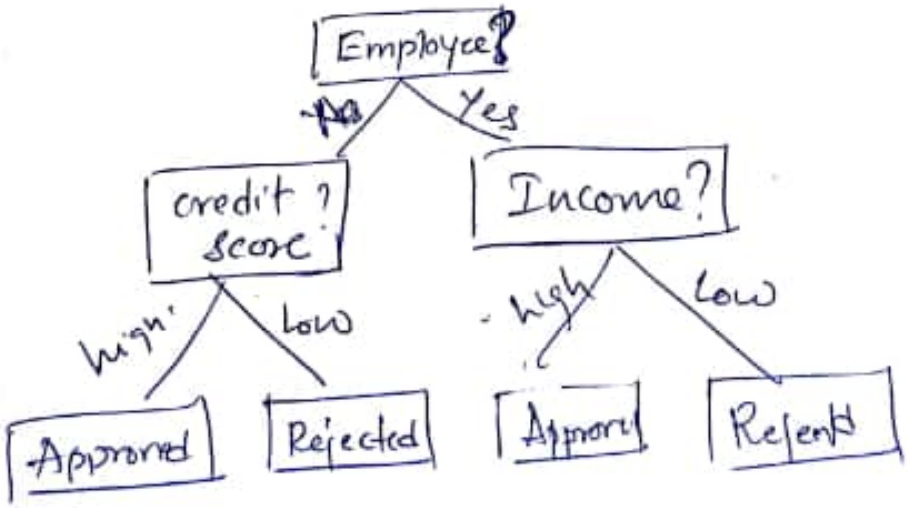

best hyperplane

UNIT-3

Decision Trees

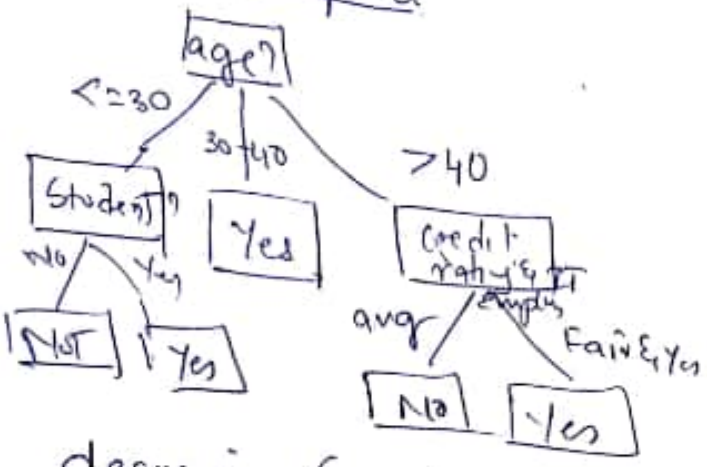
- A DT is a supervised machine learning algorithm used for classification and Regression
- It works by splitting data into smaller subsets based on feature (attribute) values and represents decisions in the form of a tree structure.
- It is a tree structured classifier
- It consists of decision nodes and leaf nodes.
  - Nodes where decisions are conducted on attributes are → decision nodes.
  - Nodes where tree stops → leaf nodes.

Eg's Loan Approved / Rejected.

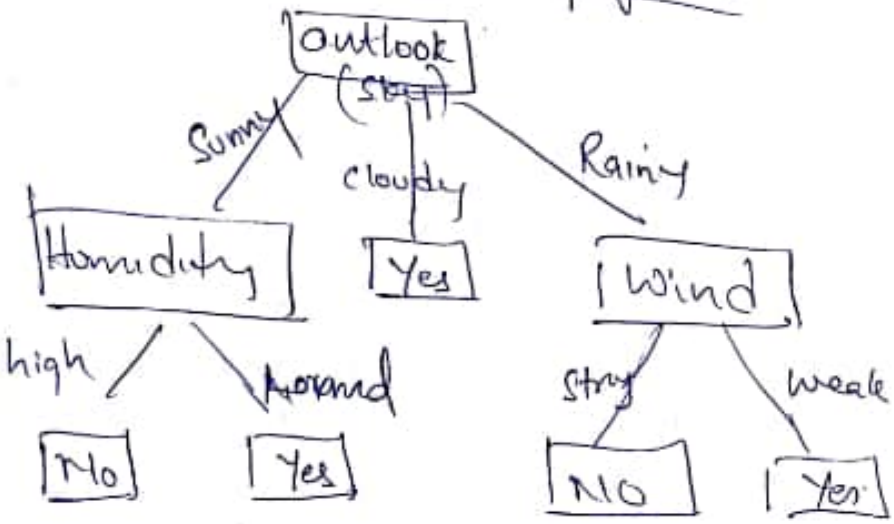


		Wind	PlayTennis
			No

Buy a computer



decision for playing games



Decision Trees are represented using above notation.

They are shown in the form of Rules

ID3 (Iterative Dichotomiser 3)

→ ML method for ~~conducting~~ constructing a decision tree, from a dataset for classification.
→ It was developed by Ross Quinlan - 1980
→ It employs a top-down, greedy approach that mainly depends on entropy and Info. Gain. to select the best attribute for splitting the data at each node.

Entropy — A measure of the impurity/randomness in a set of data.
Less entropy - pure

Information Gain — This quantifies - expected entropy that results from splitting the data on a specific attribute

ID3 (Examples, Target attribute, Attributes)

Examples are the training Examples.
Target attribute — attribute whose value is to be predicted by the tree
Attributes — list of other attributes that may be tested by the learned decision tree.

o/p
Returns a decision tree that classifies correctly the given example.

① Selecting the best Attribute
② creating Tree nodes.
③ stopping criteria.
④ Handling Missing values.
⑤ Tree Pruning

CART

→ Cart is a type of decision tree used for classification & Regression.
→ It builds a binary-tree structure
→ Cart uses Gini Index to split nodes (Impurity measure) (in classification)
→ Cart uses MSE & Variance Reduction for splitting in continuous data (numeric).
→ CART Tree construction Process
- Start with full dataset at root
- Select best feature & split point
- Split data into 2 subsets
- Repeat recursively for each child node
- stop when < Node is pure / Minimum Samples / max. depth reached

How to split an attribute using Gini Index.

→ Just like the information Gain measure, one more possibility to pick up features for the split is Gini Impurity
→ A decision tree is to have each leaf node represent data points that belong to same class, and no mismatches. This is known as 'purity'.
- If a leaf is pure, then all examples belong to one class
- To choose a split — The Algorithm loops/ repeats over different features and checks how many points belong to each class.
- CART produces Binary splits as

Feature ≤ threshold
Feature > threshold

- If a node is pure then $N(i) = 0$ for all values of i except one.
- So, for any particular feature k, we can compute

$$G_k = \sum_{i=1}^{c} \sum_{j \neq i} N(i) \cdot N(j)$$

c is the no. of classes.

Infact $\sum_{i} (N(i)) = 1$ then

$$\sum_{j \neq i} (N(j)) = 1 - N(i).$$

∴ the above equation becomes

$$G_k = 1 - \sum_{i=1}^{c} N(i)^2$$

→ The information Gain can then be measured in the same way, subtracting each value G_i from the total Gini impurity.

- We compute Gini Index of the features as below

$$\text{GiniIndex}(a_i) = \sum_{v \in \text{values}(a_i)} \frac{|S_v|}{|S|} \text{Gini}(S_v)$$

a_1	a_2	a_3	Target class
T	T	1.0	+
T	T	6.0	+
T	F	5	−
F	F	4	+
F	T	7	−
F	T	3	−
F	F	8	−
T	F	7	+
F	T	5	−

$$\text{Gini} = 1 - \sum_{i=1}^{C} P(i)^2$$

$$\text{Gini}(T) = 1 - \left(\frac{3}{4}\right)^2 - \left(\frac{1}{4}\right)^2 = 0.375$$

$$\text{Gini}(F) = 1 - \left(\frac{1}{5}\right)^2 - \left(\frac{4}{5}\right)^2 = 0.32$$

$$\text{Gini}(a_1) = \sum_{v \in \{T,F\}} \frac{|S_v|}{|S|} \text{Gini}(S_v)$$

$$\text{Gini}(a_1) = \frac{4}{9}(0.375) + \frac{5}{9}(0.32) = \underline{0.344}$$

$$\text{Gini}(a_2) = \frac{5}{9}(0.48) + \frac{4}{9}(0.5) = 0.4889$$

Smaller the Gini Index, the split is pure.
Hence a_1 is the best split attribute

Multilayer Perceptron (MLP)

A **Multilayer Perceptron** is a type of **Artificial Neural Network (ANN)** that consists of **multiple layers of neurons** and is trained using **backpropagation**.

Structure of MLP

An MLP has **three types of layers**:

1. **Input Layer**
   - Receives the input features
   - No computation is done here
2. **Hidden Layer(s)**
   - One or more layers
   - Perform nonlinear transformations
   - Use activation functions like ReLU, sigmoid, or tanh
3. **Output Layer**
   - Produces the final output
   - Activation depends on the task (e.g., sigmoid for binary classification, softmax for multi-class)

Feedforward Algorithm

Feedforward is the process of **passing input data forward** through the network to obtain the output.

Steps:

1. Initialize weights and biases (small random values)
2. Take input vector X
3. For each hidden layer:
   - Compute weighted sum:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

   - Apply activation function:

$$a^{(l)} = f(z^{(l)})$$

4. Repeat until output layer
5. Produce predicted output $\hat{y}$

Multi-Layer Perceptron

Perceptron — A Single-Layer perceptron is a feed-forward neural network consisting of input layer, output neuron, weights and a threshold function. (activation)

* Information flows only in one direction.
* It mainly classifies only linearly seperable data and the output is usually binary (0/1)

Multi-layer Perceptron (MLP)

- It is a feed-forward neural network consisting of an input layer, one or more hidden layers and an output layer.
- Unlike S.L Perceptron, a MLP can solve both linearly and non-linearly seperable problems.
- It uses the back-propagation algorithm for learning, where the error between target output and actual output is minimized using gradient descent
- The delta rule is used to update weights iteratively.
- It has no cycles / loops → purely feed forward network.
- Hidden Layers in MLP are used to learn complex, non-linear relationships in data.

